Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

The first line contains an integer representing the number of processes.

The next lines contain three integers for each process, where

he first integer denotes the Arrival Time (AT)
he second integer denotes the Burst Time (BT)
he third integer denotes the Priority of the process.

lower numerical value indicates a higher priority.

tput Format

Select Language : C (GCC 9.2.0)

```c
#include <stdio.h>

struct Process {
    int id, at, bt, priority;
    int ct, tat, wt, completed;
};

int main() {
    int n;
    printf("Enter number of processes: \n");
    if (scanf("%d", &n) != 1) return 0;

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Enter AT, BT and Priority for P%d: \n", p[i].id);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
        p[i].completed = 0;
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case

Non-Preemptive Priority Scheduling algorithm

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

Input Format

First line contains an integer representing the number of processes.

Next lines contain three integers for each process, where

First integer denotes the Arrival Time (AT)
Second integer denotes the Burst Time (BT)
Third integer denotes the Priority of the process.

Numerical value indicates a higher priority.

```
    int current_time = 0, completed_count = 0;
    float total_tat = 0, total_wt = 0;

    while (completed_count < n) {
        int idx = -1;
        int min_priority = 1e9;

        for (int i = 0; i < n; i++) {
            if (p[i].at <= current_time && !p[i].completed) {
                if (p[i].priority < min_priority) {
                    min_priority = p[i].priority;
                    idx = i;
                } else if (p[i].priority == min_priority) {
                    if (p[i].at < p[idx].at) {
                        idx = i;
                    }
                }
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Non-Preemptive Priority Scheduling algorithm

Completed

Write a C program to implement the Non-Preemptive Priority Scheduling algorithm. The program should accept number of processes their Arrival Time (AT), Burst Time (BT), and Priority for each process. The process with the highest priority (lowest numerical value) must be executed first among those that have arrived. Calculate the Average Waiting Time and Average Turnaround Time.

nput Format

The first line contains an integer representing the number of processes.

he next lines contain three integers for each process, where

he first integer denotes the Arrival Time (AT)
he second integer denotes the Burst Time (BT)
he third integer denotes the Priority of the process.

lower numerical value indicates a higher priority.

tput Format

Select Language : C (GCC 9.2.0)

```c
        if (idx != -1) {
            p[idx].ct = current_time + p[idx].bt;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;
            total_tat += p[idx].tat;
            total_wt += p[idx].wt;
            p[idx].completed = 1;
            completed_count++;
            current_time = p[idx].ct;
        } else {
            current_time++;
        }
    }

    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);

    return 0;
}
```

Test Cases 2

Run Sample Cases

Run All Cases

Custom Test Case

> Custom Test

Sample Test Case